

파이썬 Canvas 위젯

[1차시] Canvas 도화지 생성과 기초 사각형 그리기

📖 핵심 개념 설명

Tkinter 창 안에서 그림을 그릴 수 있는 전용 구역인 Canvas 위젯을 생성하는 방법을 배웁니다. 스택에 들어갈 데이터 블록을 직사각형으로 표현할 것이므로, x와 y 좌표를 지정하여 도형을 그리는 `create_rectangle` 함수의 사용법을 확실히 익혀야 합니다.

📄 전체 실습 코드

```
from tkinter import *

root = Tk()
root.title("스택 시각화 준비")

# 가로 400, 세로 500 크기의 하얀색 캔버스 도화지 생성
cvs = Canvas(root, width=400, height=500, bg="white")
cvs.pack(pady=20)

# (x1, y1)에서 (x2, y2)까지 연결되는 파란색 사각형 그리기
cvs.create_rectangle(150, 400, 250, 440, fill="skyblue", outline="black")

root.mainloop()
```

🔍 코드 훑아보기

- **`Canvas(root, width=400, height=500)`** : 메인 창(root) 안에 너비 400, 높이 500 픽셀의 그림 영역을 확보합니다.
- **`create_rectangle(x1, y1, x2, y2)`** : 왼쪽 위 꼭짓점 좌표(x1, y1)와 오른쪽 아래 꼭짓점 좌표(x2, y2)를 지정하여 직사각형을 생성합니다.

[2차시] 선과 타원 그려보기 및 속성 변경

📖 핵심 개념 설명

사각형 외에도 선과 타원을 자유롭게 그려보며 좌표계 사용에 익숙해집니다. 선의 두께를 바꾸거나 도형의 윤곽선 색상을 지정하는 옵션을 배우고, 나중에 데이터를 안전하게 담아둘 U자 모양의 스택 컨테이너 그릇을 상상하며 선 긋는 연습을 진행합니다.

📄 전체 실습 코드

```
from tkinter import *

root = Tk()
cvs = Canvas(root, width=400, height=500, bg="white")
cvs.pack(pady=20)

# 굵기가 5인 빨간색 수평선 긋기
cvs.create_line(100, 250, 300, 250, fill="red", width=5)

# 굵기가 5인 파란색 수직선 긋기
cvs.create_line(200, 100, 200, 400, fill="blue", width=5)

# 가상의 사각형 테두리에 맞춰 짝 차는 노란색 타원 그리기
cvs.create_oval(150, 200, 250, 300, fill="yellow", outline="green", width=3)

root.mainloop()
```

🔍 코드 훑아보기

- **create_line(x1, y1, x2, y2, width=5)** : 시작점(x1, y1)에서 끝점(x2, y2)을 이어주는 두꺼운 5픽셀짜리 선을 긋습니다.
- **outline='green'** : 타원이나 사각형의 테두리 선 색상을 별도로 초록색으로 지정합니다.

[3차시] 도형에 글자 새기기와 태그(tags)로 그룹 묶기

📌 핵심 개념 설명

사각형 모양의 블록만 덩그러니 있으면 어떤 데이터가 들어있는지 파악하기 어렵습니다. create_text를 이용해 화면에 글자를 직접 찍어냅니다. 또한 사각형과 글자가 한 세트로 움직이도록 tags 옵션을 이용해 두 요소에 동일한 이름표(태그)를 붙이는 아주 중요한 개념을 학습합니다.

📄 전체 실습 코드

```
from tkinter import *

root = Tk()
cvs = Canvas(root, width=400, height=500, bg="white")
cvs.pack(pady=20)

# 사각형과 텍스트를 "data_block"이라는 하나의 태그로 묶음
cvs.create_rectangle(150, 100, 250, 140, fill="lightgreen", tags="data_block")

# 사각형의 정확한 정중앙(200, 120)을 x, y로 설정하여 텍스트 배치
cvs.create_text(200, 120, text="파이썬", font=("Arial", 14, "bold"), tags="data_block")
```

```
root.mainloop()
```

🔍 코드 훑아보기

- **create_text(x, y, text='문구')** : 지정된 x, y 좌표를 텍스트의 정중앙 기준으로 삼아 글자를 예쁘게 배치합니다.
- **tags='data_block'** : 도화지 위의 여러 그림 중 특정한 것들만 그룹으로 묶어 나중에 한꺼번에 조종할 수 있게 해주는 이름표입니다.

[4차시] 버튼으로 Canvas 위젯 움직이기 (move)

📌 핵심 개념 설명

고정된 그림이 아니라 살아 움직이는 화면을 구성합니다. 버튼을 누를 때마다 실행되는 콜백 함수 안에서 Canvas의 move 함수를 사용합니다. 이때 이동시키고 싶은 태그 이름과 x축, y축 이동 픽셀을 입력하면 태그로 묶인 그림 전체가 부드럽게 이동합니다.

📄 전체 실습 코드

```
from tkinter import *

def move_block():
    # "data_block" 태그를 가진 모든 그림을 x축으로 0, y축으로 50 픽셀만큼 이동
    cvs.move("data_block", 0, 50)

root = Tk()
cvs = Canvas(root, width=400, height=500, bg="white")
cvs.pack()

cvs.create_rectangle(150, 50, 250, 90, fill="orange", tags="data_block")
cvs.create_text(200, 70, text="스택 이동", font=("Arial", 12, "bold"),
tags="data_block")

btn = Button(root, text="블록을 아래로 떨어뜨리기", command=move_block)
btn.pack(pady=10)

root.mainloop()
```

🔍 코드 훑아보기

- **cvs.move('태그이름', dx, dy)** : 해당 태그를 가진 모든 객체를 현재 위치에서 가로로 dx, 세로로 dy만큼 상대 이동시킵니다.

- **command=move_block** : 버튼을 클릭할 때마다 도형이 y축 양의 방향(아래쪽)으로 50픽셀씩 계속 이동합니다.

[5차시] 마우스 이벤트와 도화지 연동하기 (bind)

📌 핵심 개념 설명

버튼 위젯이 아니라 Canvas 도화지 자체를 마우스로 직접 클릭했을 때 반응하도록 만듭니다. bind 함수를 사용해 사용자의 클릭 이벤트를 감지하고, 이벤트 객체에서 클릭한 화면의 정확한 x, y 좌표를 얻어와 그 위치에 실시간으로 새로운 도형을 만들어봅니다.

📄 전체 실습 코드

```
from tkinter import *

def draw_click_box(event):
    # 사용자가 마우스를 클릭한 위치의 좌표를 가져옴
    click_x = event.x
    click_y = event.y

    # 클릭한 좌표를 중심으로 가로세로 50짜리 보라색 사각형 생성
    cvs.create_rectangle(click_x - 25, click_y - 25, click_x + 25, click_y + 25,
fill="purple")

root = Tk()
root.title("클릭 이벤트 실습")

cvs = Canvas(root, width=400, height=500, bg="lightyellow")
cvs.pack(pady=20)

# 마우스 왼쪽 버튼 클릭 이벤트를 감지하여 draw_click_box 함수 즉시 실행
cvs.bind("<Button-1>", draw_click_box)

root.mainloop()
```

🔍 코드 훑아보기

- **event.x, event.y** : 마우스를 클릭한 순간의 정확한 화면 좌표값을 프로그램으로 가져옵니다.
- **cvs.bind('<Button-1>', 함수)** : <Button-1>은 마우스 왼쪽 버튼을 의미하며, 이 버튼이 클릭될 때 지정한 함수를 즉시 호출합니다.

[6차시] 스택 블록 생성 자동화 (함수 모듈화)

🔑 핵심 개념 설명

데이터가 추가될 때마다 사각형을 그리고 텍스트를 입히는 코드를 매번 두 줄씩 작성하면 코드가 너무 복잡해집니다. 이를 하나의 공장(함수)으로 묶고, x좌표, y좌표, 텍스트 내용, 태그 이름을 매개변수로 전달받아 알아서 블록 한 세트를 깔끔하게 찍어내는 모듈 함수를 설계합니다.

📄 전체 실습 코드

```
from tkinter import *

root = Tk()
cvs = Canvas(root, width=400, height=500, bg="white")
cvs.pack(pady=20)

# 블록 생성 전용 공장 함수
def create_stack_block(x, y, data_text, tag_name):
    # 기준점(x, y)으로부터 가로 100, 세로 40 크기의 사각형 생성
    cvs.create_rectangle(x, y, x + 100, y + 40, fill="pink", tags=tag_name)
    # 사각형 정중앙 좌표(x+50, y+20)에 텍스트 쓰기
    cvs.create_text(x + 50, y + 20, text=data_text, font=("Arial", 12, "bold"),
tags=tag_name)

# 함수를 호출하여 화면에 3개의 데이터 블록을 차곡차곡 쌓아보기
create_stack_block(150, 400, "첫번째", "block_0")
create_stack_block(150, 360, "두번째", "block_1")
create_stack_block(150, 320, "세번째", "block_2")

root.mainloop()
```

🔍 코드 훑아보기

- **def create_stack_block** : 도형의 반복 생성을 담당하는 아주 중요한 모듈화 과정입니다. 스택 시뮬레이션의 핵심 부품이 됩니다.
- **x + 50, y + 20** : 블록이 어디에 생성되든 글자가 항상 블록 정중앙에 위치하도록 기준점(x, y)에 비율을 더해줍니다.